

GPU / cuTile · Projekt B

Auto-Tuning für Tensor-Kontraktionen

Aus Einsum-String + Shapes automatisch eine gute cuTile-Tiling-Config finden — gemessen, nicht geraten.

Niklas Becker-Klöser · Daria Elagina

cmk,ckn→cmn · acspx,bspy→abcyx · NVIDIA GB10 (DGX Spark)

Von Hand getunt skaliert nicht

- ▶ A05 & A06: L2-optimale Aufteilung von Hand hergeleitet.
- ▶ Für jede neue Kontraktion / Shape / GPU: neu nachdenken.
- ▶ Ziel: Eingabe = Einsum + Shapes → Ausgabe = gute Config.
- ▶ Kein allgemeiner Tensor-Compiler — zwei Familien, Configs getunt.

Zwei Testfälle

A05 · **batched Matmul**

cmk, ckn → cmn

A06 · **Tensor-Ring**

acsp_x, bsp_y → abcy_x

Die Performance steckt in der Config, nicht im Kernel-Code — genau das automatisieren wir.

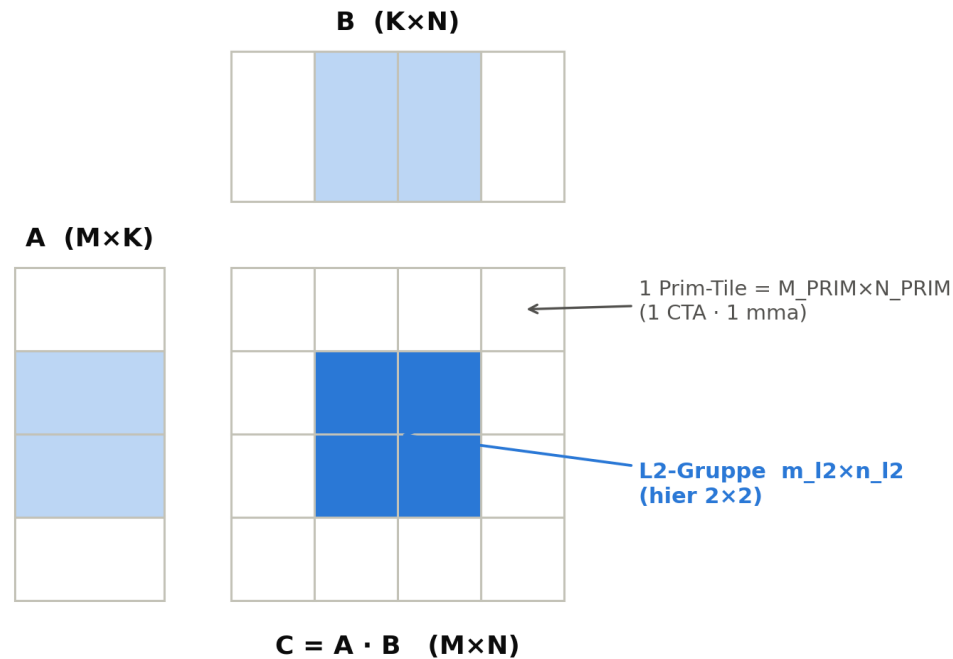
TEIL 1

Umsetzung & Architektur

Pipeline · Suchraum · Kernel · Pruning · A06-Transfer

Was ist eine Tiling-Config?

L2-Reuse: eine Block-Gruppe teilt sich A-Zeilen und B-Spalten



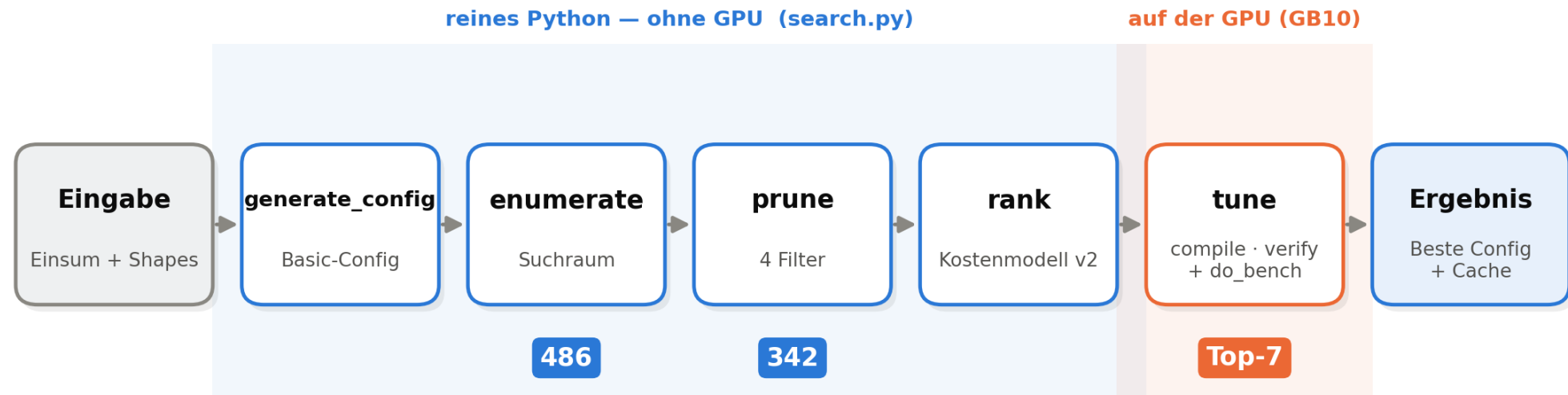
Die Gruppe lädt ihre A-Zeilen und B-Spalten je einmal und nutzt sie mehrfach → bleibt im L2 resident

- ▶ Prim-Tiles $M/N/K_{\text{PRIM}}$: die mma-Kachel eines CTA.
- ▶ L2-Gruppe $m_{l2} \times n_{l2}$: zeitlich nahe Blöcke → L2-Reuse.
- ▶ Exec-Reihenfolge PAR | SEQ | PRIM (K nie PAR).
- ▶ Variante A = Swizzle, B = SEQ-Loops.

L2-Reuse entsteht durch die zeitliche Block-Gruppe, nicht durch eine räumliche Kachel.

Die Tuner-Pipeline

Die Tuner-Pipeline: von Einsum zur gemessen besten Config



generate → enumerate → prune → rank offline (reines Python); nur die Top-7 werden auf der GPU gemessen.

Der Suchraum: die Knöpfe

- ▶ $M_{\text{PRIM}}, N_{\text{PRIM}} \in \{64, 128, 256\}$
- ▶ $K_{\text{PRIM}} \in \{32, 64, 128\}$
- ▶ $M_{\text{L2}}, N_{\text{L2}} \in \{2, 4, 8\}$
- ▶ Variante $\in \{A, B\}$

$$3 \cdot 3 \cdot 3 \cdot 3 \cdot 3 \cdot 2 = 486$$

Kandidaten (nicht die „81“ aus dem Pitch)

✓ Akzeptanztest

Die handoptimierte A05-Config ist im Set — sonst könnte der Tuner sie nie finden.

Bewusst klein & hardware-sinnvoll gehalten; krumme Shapes werden hochgepadet (PaddingMode.ZERO).

Ein generischer Kernel — kein String-Codegen



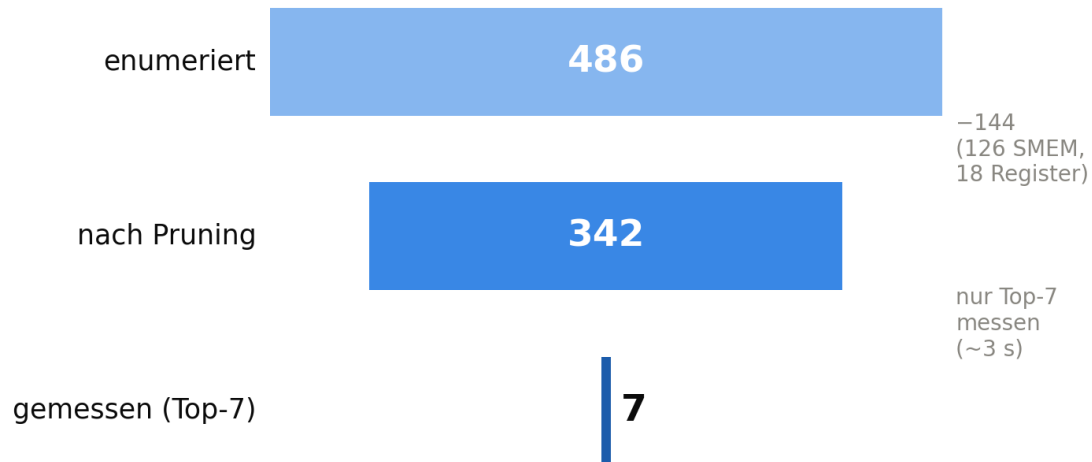
```
@ct.kernel
def matmul_variant_a(A, B, C,
    M_PRIM: ct.Constant[int], N_PRIM: ct.Constant[int],
    K_PRIM: ct.Constant[int], M_L2: ct.Constant[int],
    N_L2: ct.Constant[int], num_k_outer: ct.Constant[int]):
    pid = ct.bid(0) # eine Block-ID ...
    n_l2_idx = pid % N_L2; pid //= N_L2 # ... in die
    m_l2_idx = pid % M_L2; pid //= M_L2 # L2-Gruppe swizzeln
    acc = ct.zeros((M_PRIM, N_PRIM), dtype=ct.float32)
    for k_it in range(num_k_outer):
        a = ct.load(A, ..., padding_mode=ct.PaddingMode.ZERO)
        b = ct.load(B, ..., padding_mode=ct.PaddingMode.ZERO)
        acc = ct.mma(a, b, acc) # Tensor-Core-Kachel
    ct.store(C, ..., tile=acc.astype(ct.float16))
```

- ▶ Tile-Größen als `ct.Constant` → JIT spezialisiert pro Wert.
- ▶ Keine fragilen String-Templates / `exec()`.
- ▶ Variante A: Swizzle über die Block-ID.
- ▶ Korrektheit gegen `torch.einsum` (`allclose`).

Compile ~0.4 s pro Config, auf der GB10 verifiziert — der ganze Ansatz hängt an dieser Spezialisierung.

Statisch eingrenzen — und wie weit wirklich

Suchraum eingrenzen: 486 → 342 → 7 (A05)



```
def prune_reason (cand, dev, ...):
    if cand.m_prim % 16 or cand.n_prim % 16 or cand.k_prim % 16:
        return 'mma_align' # 1) MMA-Teilbarkeit
    if estimate_smem_bytes (cand) > smem_limit:
        return 'smem_exceeded' # 2) Shared-Memory-Budget
    if estimate_acc_registers (cand) > 0.5 * dev.regs_per_block:
        return 'acc_registers' # 3) Akku-Register
    if padding_ratio (cand) > max_padding:
        return 'padding_waste' # 4) Padding-Verschwendung
    return None # -> Kandidat ueberlebt
```

Was Filter 2 & 3 rechnen (Formeln → nächste Folie)

SMEM $(m \cdot k + k \cdot n) \cdot 2 \cdot B \cdot 2 \text{ Stages} \leq 100 \text{ KB} \rightarrow 126 \text{ raus}$
 Akku $m_prim \cdot n_prim \leq \frac{1}{2} \cdot 65536 \text{ (fp32-Reg)} \rightarrow 18 \text{ raus}$

Nur 486→342: SMEM hängt nur an Prim-Größen — m_l2/n_l2 & Variante muss man messen (25 MB L2 killt die L2-Regel).

Was Pruning und Kostenmodell konkret rechnen

Pruning — 4 statische Filter (ohne Compile) Ranking — DRAM-Traffic / Bandbreite

1 MMA-Teilbarkeit

$m_{\text{prim}}, n_{\text{prim}}, k_{\text{prim}} \bmod 16 = 0$
fp16-Tensor-Cores brauchen 16er-Vielfache

2 SMEM-Budget (harter Filter)

$(m \cdot k + k \cdot n) \cdot 2 \text{ Byte} \cdot 2 \text{ Stages} \leq 100 \text{ KB}$
fp16-Operanden, double-buffered · 101376 – 1024 → 126 raus

3 Akku-Register

$m_{\text{prim}} \cdot n_{\text{prim}} \leq \frac{1}{2} \cdot 65536 \quad (1 \text{ Reg/Elem.})$
fp32-Akku liegt in Registern → 18 raus

4 Padding-Verschwendung

$V_{\text{padded}} / V_{\text{orig}} \leq 8$
gepadding gegen Original-Volumen

geschätzter DRAM-Traffic (worst case, × 2 Byte):

A: $M \cdot K \cdot \text{ceil}(N / (n_{\text{l2}} \cdot n_{\text{prim}}))$

B: $K \cdot N \cdot \text{ceil}(M / (m_{\text{l2}} \cdot m_{\text{prim}}))$

C: $M \cdot N$

größere L2-Gruppe $m_{\text{l2}} \cdot n_{\text{l2}} \rightarrow$ A/B seltener nachladen →
weniger Traffic (das ist der L2-Reuse im Modell)

$t_{\text{mem}} = \text{Traffic} / \text{BW}$

$\text{BW} = \text{mem_clk} \cdot (\text{Bus} / 8) \approx 273 \text{ GB/s (GB10)}$

Roofline: $t = \max(t_{\text{mem}}, t_{\text{compute}})$

$t_{\text{compute}} = 2 \cdot M \cdot N \cdot K / (\text{Peak} \cdot \text{util})$

$\text{Peak} = \text{SMs} \cdot \text{Takt} \cdot \text{FMAs} \cdot 2 \approx 119 \text{ TFLOPS (GB10)}$

→ **max()** schaltet das Regime selbst um (device_props)

Alles aus device_props ausgelesen (SMEM / Register / Bandbreite / Peak) — nur Tile-Form, k_{prim} und L2-Gruppe variiert der Tuner.

A06: eine zweite Struktur-Familie

```

@ct.kernel
def matmul_ring_a(A, B, C, M_PRIM: ct.Constant[int], ...,
                  SIZE_B: ct.Constant[int], SIZE_S: ct.Constant[int]):
    pid = ct.bid(0)                                # unabhaengige Batches:
    b_idx = pid % SIZE_B; pid //= SIZE_B           # a,c nur in A, b nur in B
    c_idx = pid % SIZE_C; pid //= SIZE_C
    a_idx = pid
    acc = ct.zeros((M_PRIM, N_PRIM), ct.float32)
    for s_it in range(SIZE_S):                      # 2. Reduktion s als SEQ-Loop
        for k_it in range(num_k_outer):             # p als prim_k im mma
            tA = ct.load(A, index=(a_idx, c_idx, s_it, k_it, ...))
            tA = ct.permute(tA, (1, 0))              # Layout ist nicht mma-fertig
            tB = ct.load(B, index=(b_idx, s_it, k_it, ...))
            acc = ct.mma(tA, tB, acc)
    ct.store(C, index=(a_idx, b_idx, c_idx, y_block, x_block), ...)

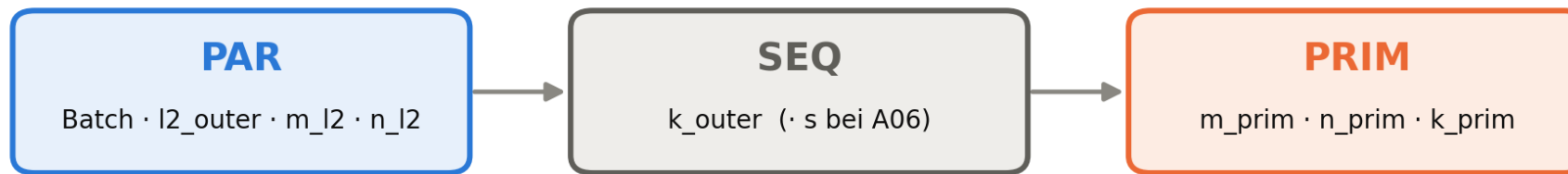
```

- ▶ Nicht „doppeltes K" — die Batch-Topologie ist das Problem.
- ▶ A05: geteilter Batch c. A06: unabhängige a,c (A) und b (B).
- ▶ → zweiter Kernel-Typ (Ring), kein Umbau.
- ▶ Der Tuner sucht Configs, nicht Kernel — A05-Pfad bleibt bitgleich.

Wie cuBLAS/CUTLASS (Template-Menge) oder Triton (autotune pro @jit): neue Topologie = neues Template, Configs getunt.

Wie wir die Reihenfolge der Dimensionen definiert haben

1 · Reihenfolge in der Config (verify):



K nie PAR · PRIM ganz rechts = je $\geq 1 \times M, N$ und K (die mma-Kachel)

2 · Split je Dimension (der Tuner variiert nur die Größen):

M	→	m_l2_outer	·	m_l2	·	m_prim	grau = Grid (l2_outer)
N	→	n_l2_outer	·	n_l2	·	n_prim	blau = L2-Gruppe (m_l2 / n_l2)
K	→	k_outer	·	—	·	k_prim	orange = PRIM (mma-Kachel)

Variante A: m_l2/n_l2 = PAR (Swizzle)

Variante B: m_l2/n_l2 = SEQ-Loop

Jede Config ist by construction gültig: die Reihenfolge PAR | SEQ | PRIM steht fest (verify), der Tuner variiert nur die Split-Größen und A/B.

TEIL 2

Evaluation & Ergebnisse

Gegen Handkernel · gegen cuBLAS · über Shapes & GPUs

Benchmark-Setup: acht Shape-Regime je Familie

Acht Shape-Regime · einsum cmk, ckn → cmn

Regime	C	M	N	K	testet
square · b4	4	4096	4096	4096	Referenz (Heimvorteil)
square · b1	1	4096	4096	4096	ohne Batch
tall M≫N	1	8192	1024	4096	rechteckig, viele Zeilen
wide N≫M	1	1024	8192	4096	rechteckig, viele Spalten
small_k	1	4096	4096	512	kleines K → bandbreiten-nah
large_k	1	1024	1024	8192	großes K → compute-nah
krumm	2	1500	3000	1000	unteilbar → Padding-Pfad
batch16	16	1024	1024	1024	viele kleine Batches

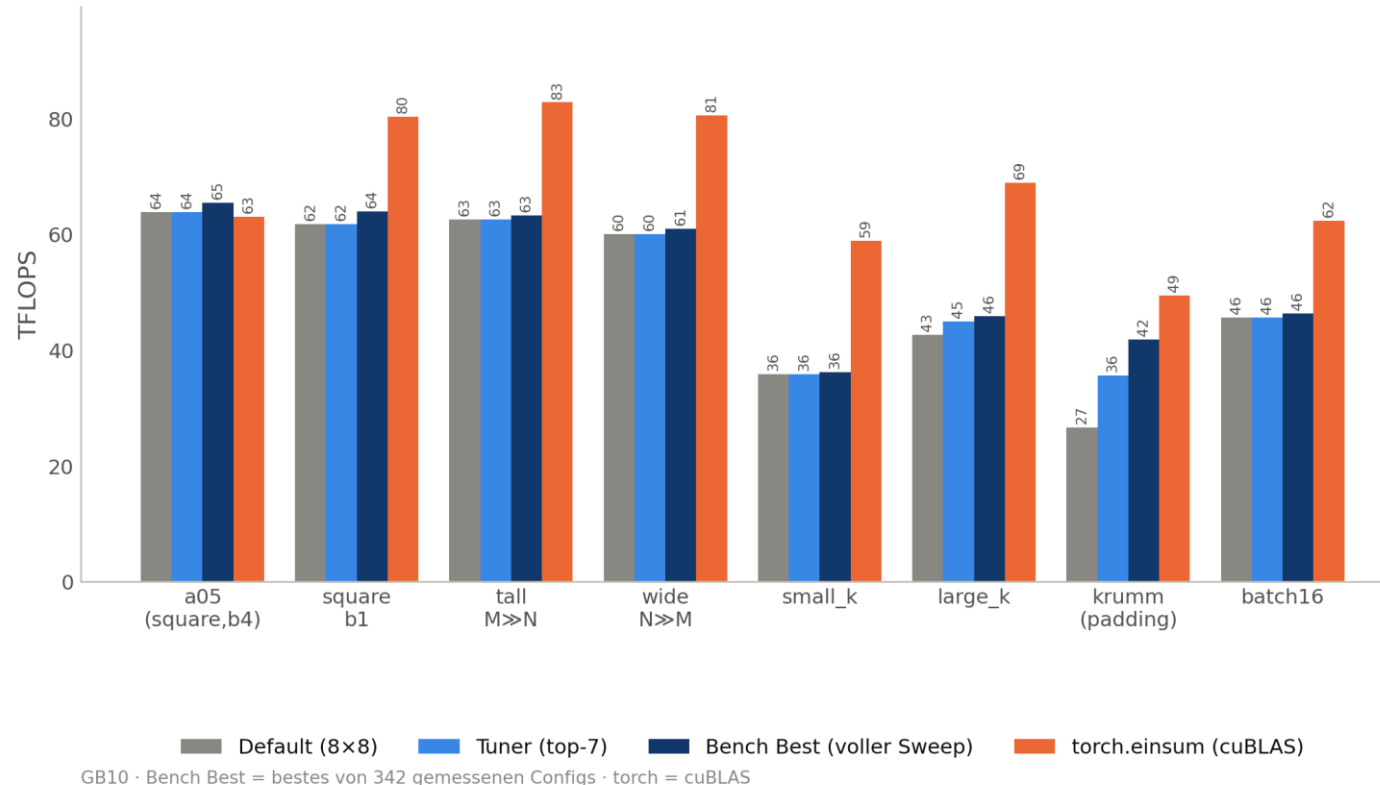
A06 nutzt dieselben acht Regime in Ring-Form (acsp_x, bsp_y → abcy_x)

- ▶ Pro Shape: Tuner vs. Default (8×8) vs. torch.einsum; A06 auch vs. Handkernel.
- ▶ fp16 rein, fp32 Akku; jede Config gegen torch.einsum geprüft (allclose).
- ▶ 8×342 (A05) + 8×171 (A06): 0 Fehlschläge, inkl. Padding-Pfad (krumm).
- ▶ GPU: NVIDIA GB10 — 48 SMs, 25 MB L2, integrierter LPDDR.

Acht Regime decken square / rechteckig / K-Extreme / unteilbar / Batch ab — fair gegen drei Referenzen, alles korrekt.

Der Tuner bestätigt die Handarbeit

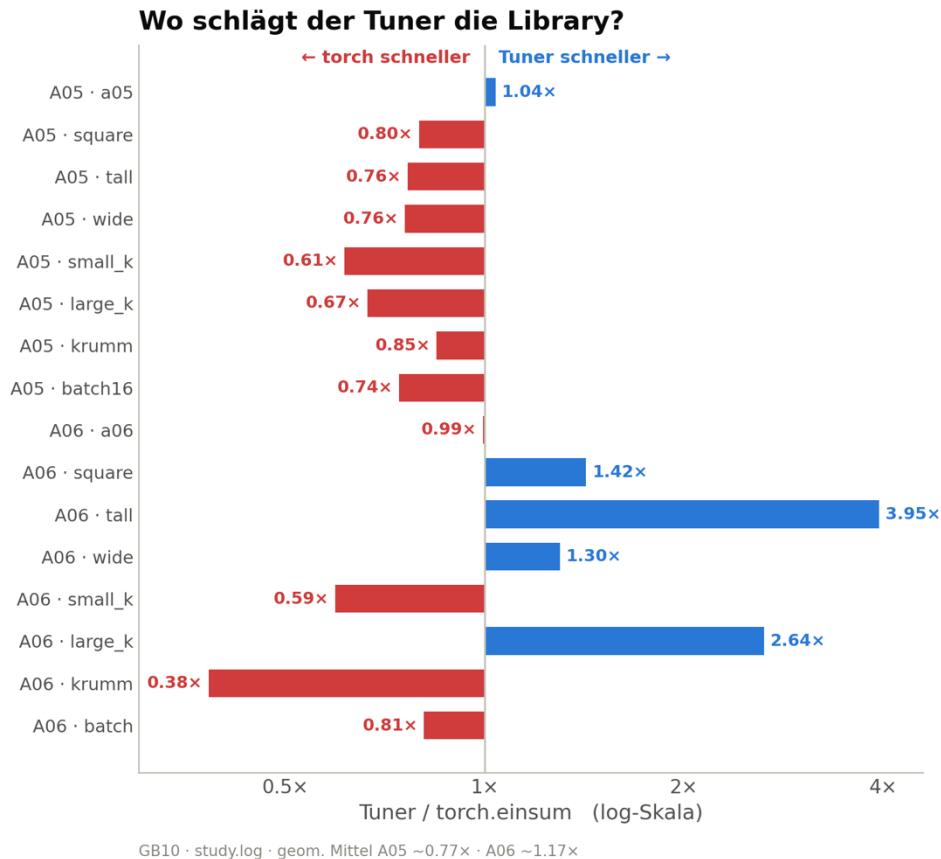
A05: Tuner-Pick $\approx$ Default — selbst die Bench-Best bleibt unter cuBLAS



- ▶ Tuner-Pick (top-7) $\approx$ Default $\rightarrow$ bestätigt die Hand.
- ▶ Bench-Best (von 342) nur knapp drüber; krumm +58 %.
- ▶ Aber: selbst die Bench-Best bleibt auf GEMM unter cuBLAS.

Warum verliert selbst unsere Bench-Best gegen torch? cuBLAS ist eine gereifte GEMM-Library (hand-optimiertes SASS, Split-K, Pipelining) — das schlägt ein einfaches Template.

Tuner vs. cuBLAS / torch.einsum

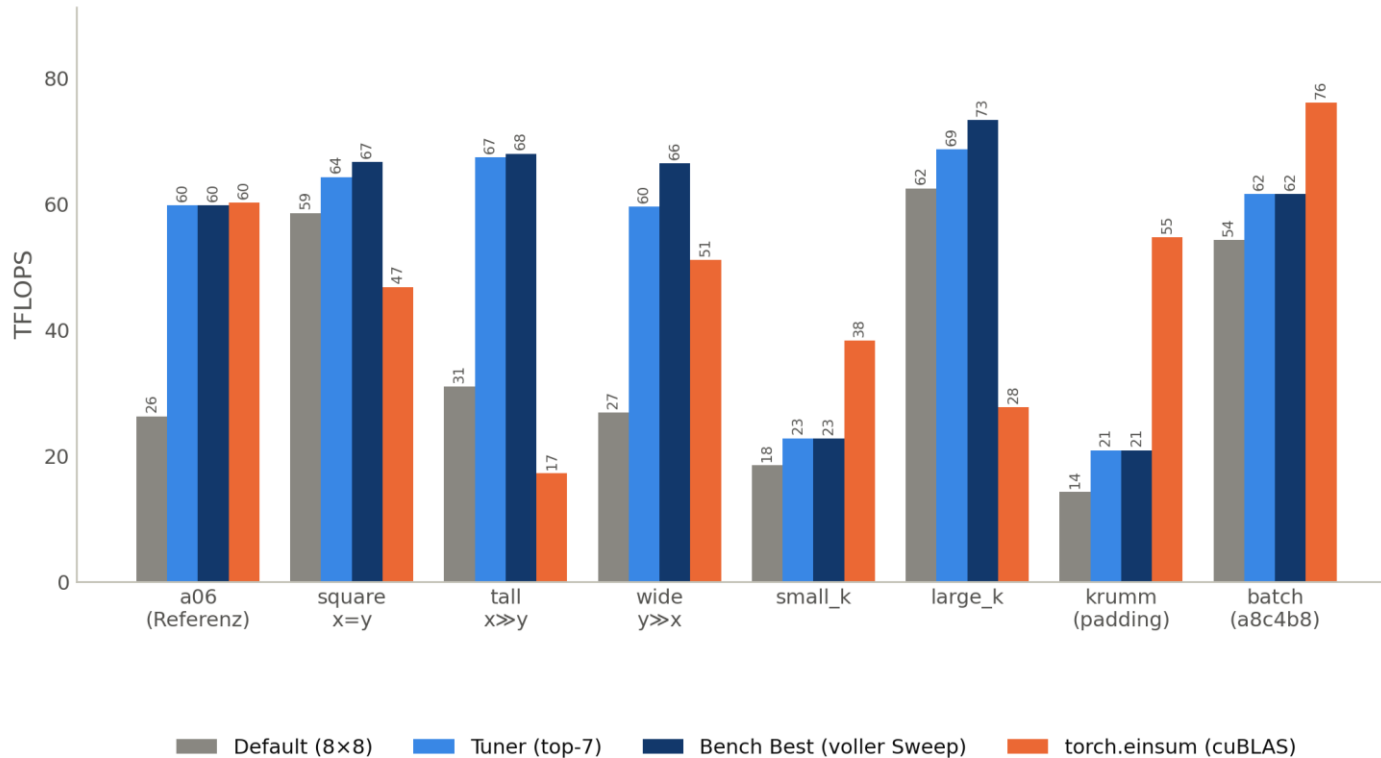


- ▶ GEMM (A05): cuBLAS gewinnt (Tuner ~77 % von torch) — erwartbar.
- ▶ Ring (A06): im Mittel ~1.17x vorn, sehr shape-abhängig.
- ▶ Großer Gewinn, wo torchs Pfad schlecht ist (tall 3.95x, large_k 2.65x).
- ▶ A06_TORCH=16.18 aus dem Assignment ist veraltet → frisch 60.2.

Der Wert ist nicht „schneller als alles“, sondern: ohne Handarbeit über beliebige Kontraktionen brauchbar — und stark, wo die Library keinen guten Pfad hat.

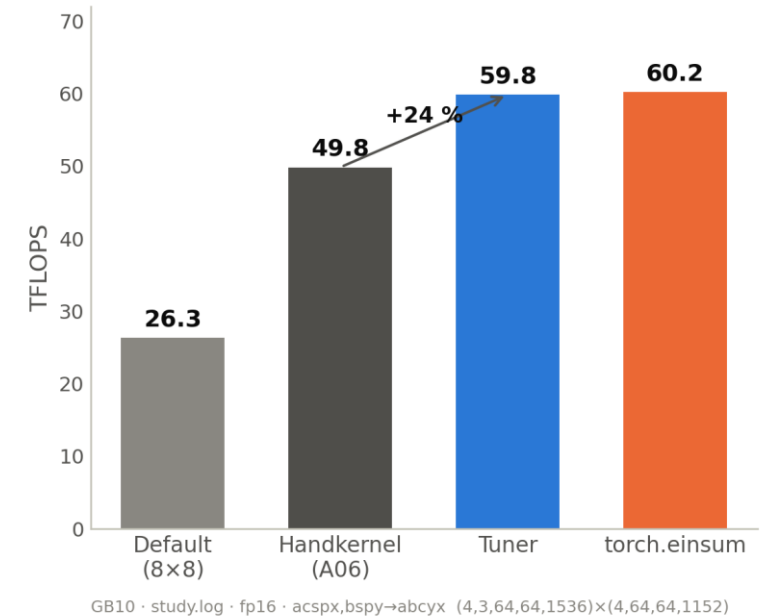
Transfer gelingt — Tuner schlägt den Handkernel

A06: Tuner » Default — Bench-Best schlägt torch, wo dessen Pfad schlecht ist



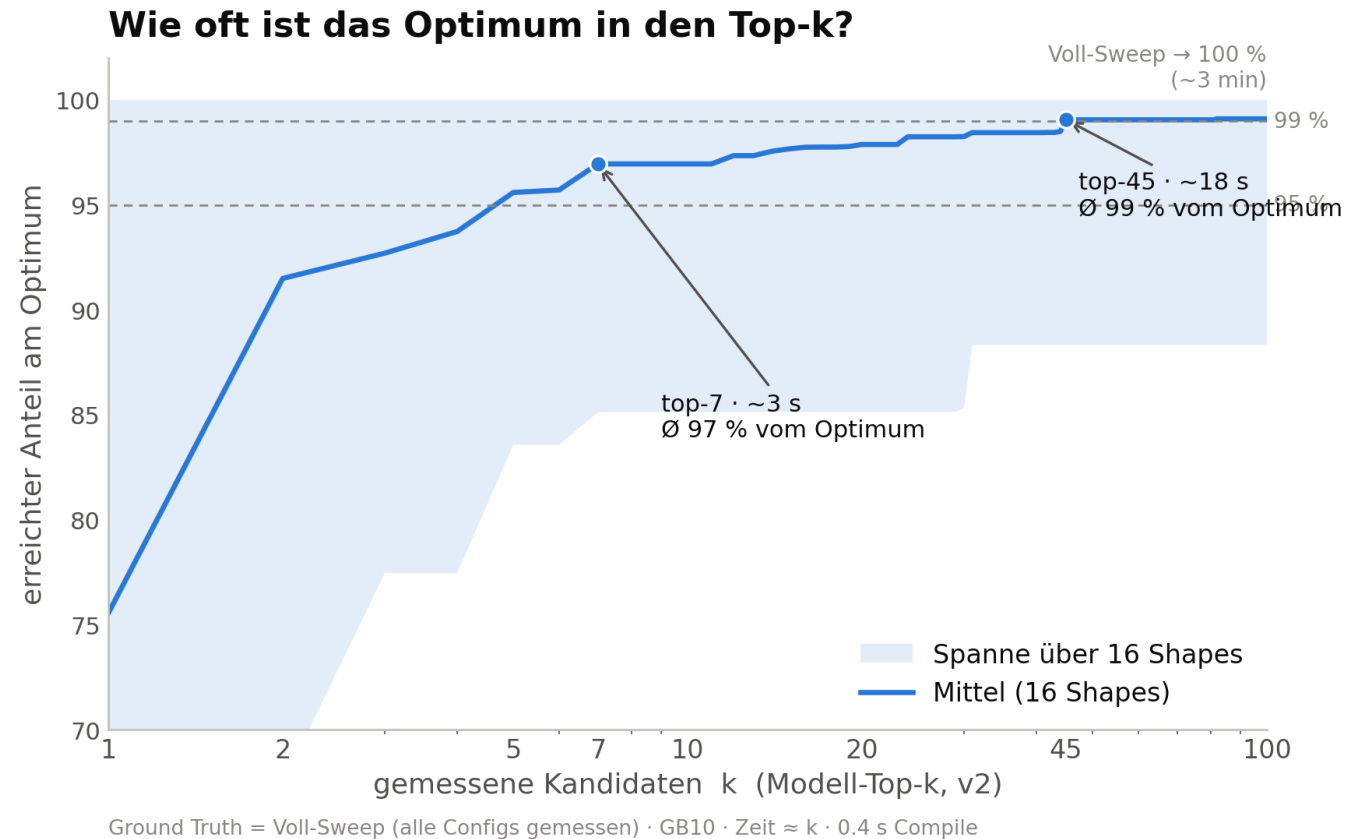
GB10 · Bench Best = bestes von 171 gemessenen Configs · torch = cuBLAS

A06-Referenz: der Tuner schlägt den Handkernel und liegt gleichauf mit torch



Anders als GEMM: unsere Bench-Best schlägt torch, wo dessen Ring-Pfad schlecht ist (tall, large_k); der Tuner schlägt zudem den Handkernel (+24 %, aus k_prim=32).

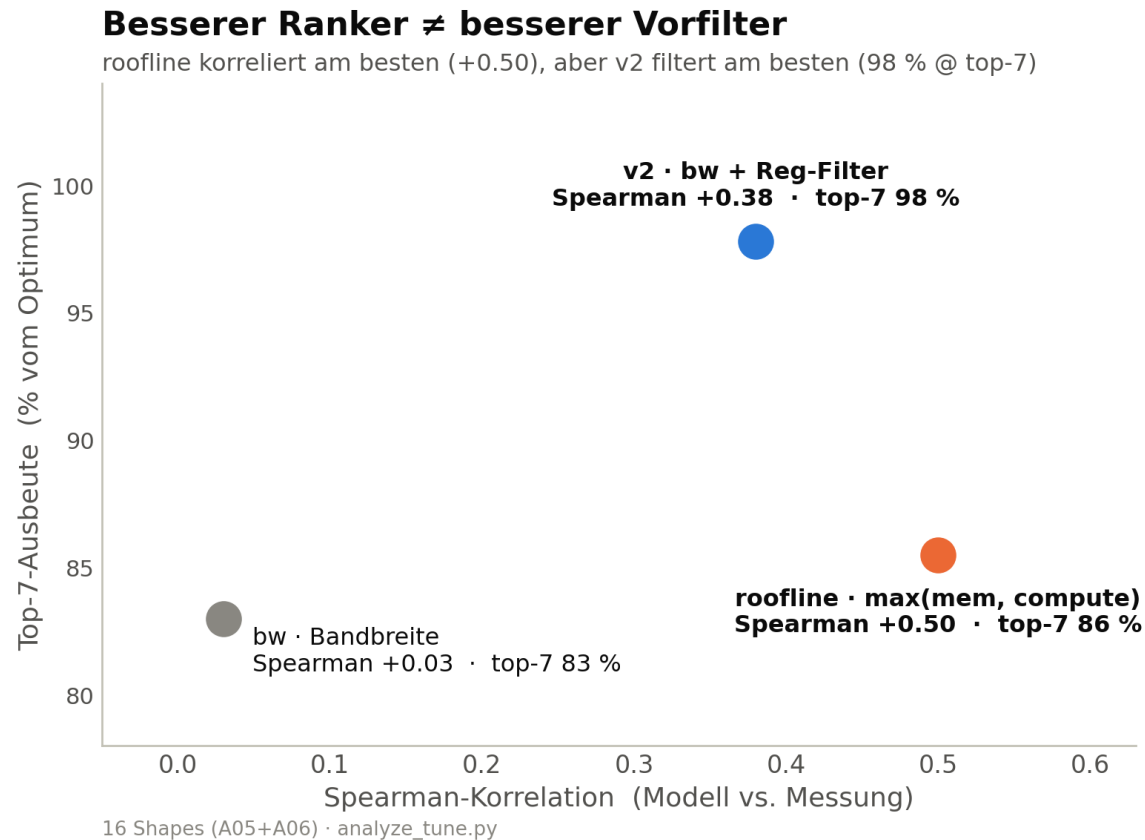
Wie oft ist das Optimum in den Top-k?



- ▶ **top-7 · ~ 3 s · $\geq 95\%$**
- ▶ **top-45 · ~ 18 s · $\geq 99\%$**
- ▶ **Voll-Sweep · ~ 3 min · 100%**
- ▶ Exakt-Beste sitzt tief — Spitzenfeld liegt im Messrauschen.

Als Vorauswahl reicht das Modell fast immer; als exakter Treffer selten — top-7 ist der Sweet Spot.

Besserer Ranker $\neq$ besserer Vorfilter

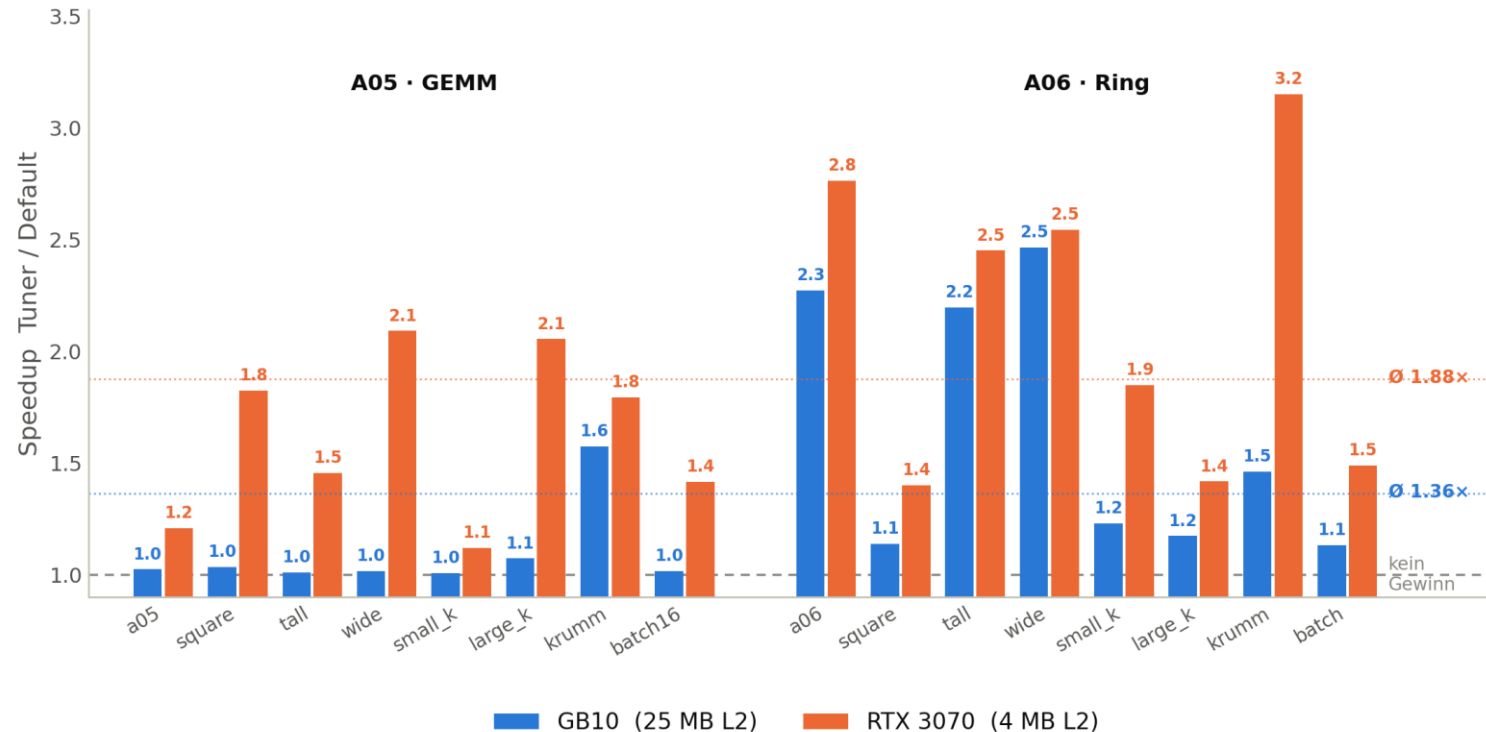


- ▶ bw (Bandbreite): falsche Physik — 25 MB L2.
- ▶ v2 (bw + Reg-Filter): 97.8 % top-7 → Default.
- ▶ roofline: beste Korrelation, schaltet Regime selbst um.

Höchste Korrelation $\neq$ beste Top-k-Ausbeute — für die Praxis zählt der Vorfilter, also v2.

Derselbe Hebel, aber andere Config pro GPU

Der Tuning-Hebel wirkt auf beiden GPUs — auf der 3070 stärker



relativer Speedup (unitless) — absolute TFLOPS beider Karten sind nicht vergleichbar · beste Config in 16/16 Shapes je GPU verschieden

Tuning hilft auf beiden Karten (Ø 1.36× GB10, 1.88× 3070) und die optimale Config ist in 16/16 Shapes verschieden — genau deshalb: GPU-spezifisch tunen + cachten.

Was der Tuner pro GPU wählt — und was es kostet

Optimale Config unterscheidet sich pro GPU (m/n/k-Prim · L2-Gruppe)

Shape	GB10 (25 MB L2)	RTX 3070 (4 MB L2)
<i>A05 · GEMM</i> a05	128/128/64 8×4	64/128/64 8×2
tall	64/256/64 2×4	64/128/64 2×2
small_k	64/256/64 4×2	128/128/32 8×2
krumm	128/128/64 2×2	256/64/32 2×8
<i>A06 · Ring</i> a06	128/128/32 4×2	128/64/64 2×2
tall	128/128/32 2×4	64/128/64 2×2
large_k	128/128/32 8×4	64/128/32 8×2
krumm	64/256/64 8×4	64/64/128 8×4

Muster: GB10 → große 128×128-Tiles (großes L2 verträgt sie) · 3070 → kleineres k_prim=32, oft asymmetrisch/64-breit

- ▶ Optimum in 16/16 Shapes verschieden — pro GPU eine andere Config.
- ▶ Autotuner (v2, top-7) trifft ø 96 % (GB10) / 90 % (3070) des Optimums.
- ▶ Kosten: GB10 ~3 s/Shape; 3070 (WSL-Compile) ~3 s–2 min — Compile dominiert.
- ▶ Roofline schaltet sein Regime selbst um; praktisch bleibt v2 der beste Vorfilter.

Pro GPU von Hand zu tunen ist nicht machbar — das nimmt der gecachte Tuner ab, ohne auf eine Karte zu overfitten.

Wann sich Tuning lohnt: Cache & Amortisierung

- ▶ Eine Kontraktion läuft ~8 ms; Tuning ~3 s ist ein Einmalposten.
- ▶ Im Netz sind Layer-Dims fix → millionenfach dieselbe Shape.
- ▶ Config-Cache: Key = Einsum + Shapes + GPU-Modell.
- ▶ Genau so machen es cuBLAS, Triton (autotune), torch.compile.

~3 s → 0 s

erste getunte Shape → jede weitere aus dem Cache

End-to-end bestätigt

16 Shapes getunt & gecacht, 2. Aufruf Cache-Hit, Top-7 bei ~95–100 %.

Break-even in Sekunden bei fester Shape; für stark dynamische Shapes hilft Bucketing/Padding.

Eingrenzen ist sicher — entscheiden muss man messen

- ▶ **A05:** Tuner reproduziert Hand ($\emptyset$ 1.03×), gewinnt bei krummen Shapes (+58 %).
- ▶ **A06:** sauberer Transfer, ein zusätzlicher Kernel-Typ, schlägt den Handkernel (+24 %).
- ▶ **cuBLAS:** auf GEMM nicht geschlagen — Wert ist Allgemeinheit + Gewinne ohne guten Library-Pfad.
- ▶ **Modell:** v2 bester Vorfilter (97.8 % @ top-7), roofline bester Ranker — messen bleibt entscheidend.
- ▶ **Praxis:** GPU-spezifisch + gecacht — auf GB10 und 3070 bestätigt: Hebel wirkt, beste Config je GPU verschieden.